**BUS5PA Predictive Analytics**

**Assessment 2 Building and Evaluating Predictive Models using SAS Enterprise Miner**

**Anh Khoa Mai – 22922993**

# Case Study: Identifying High-Severity Road Crashes in Victoria

## Table of Contents

Q1. Project Setup and Exploratory Analysis....2
Prepare the data for a credible model....2
Exploratory assessment....4
Design a validation strategy....7
Q2. Establishing a Baseline and Initial Candidate Model....8
Set up the baseline....8
Build an initial candidate model....9
Decide whether the candidate improves on the baseline....12
Q3. Developing a Contrasting Modelling Strategy....14
Choose a different model family....14
Configure and assess the contrasting model....15
Compare the modelling strategies....17
Translate the findings for decision-makers....19
Q4. Challenging and Refining the Modelling Strategy....20
Diagnose a weakness in the current modelling strategy....20
Design and implement a defensible extension....20
Judge whether the improvement is worth retaining....21
Q5. Model Comparison, Scoring and Recommendation....21
Compare all models....21
Score the selected model....23
Business recommendation....23

# Q1. Project Setup and Exploratory Analysis

## Prepare the data for a credible model

**Intended prediction point**

The prediction point is immediately after a crash has been reported to police and the initial attending-officer record has been created before any hospital, coronial, or follow-up investigation outcome is known. The task explicitly asks us to predict severity among crashes that have already been reported. This means: The unit of analysis is a reported crash event, so all pre-crash and at-scene variables such as time, location, road environment, crash type, vehicles/people involved are legitimately known at this point. This is because they describe the circumstances of the crash itself, not its aftermath; The model's operational use case is triage/resource allocation and evidence-building for treatments. This only makes sense if the inputs are things known at or near the time of reporting, not things only known afterward.

**Data leakage identified and handling**

| Group | Leakage risk | Decision |
|---|---|---|
| Total people count split by injury severity | Direct leakage | Dropped |
| Crash ID | No predictive content | Dropped |
| DCA | Borderline | Kept |
| Post-crash administrative fields | Target leakage | Dropped |
| Number of vehicles / people involved | Not leakage | Kept |

Principle: for every variable, I asked "would this be knowable from the attending officer's crash report form, independent of how the injuries eventually got coded?". If the answer was no, or only conditionally yes, the variable was dropped.

**Variable-role and measurement-level decisions**

- *Target (TargetSevere)*: binary/nominal, 1 = fatal or serious injury, 0 = other injury. Treated as a classification target, not ordinal. The case study does not ask us to distinguish fatal from serious, so collapsing them is appropriate and matches the stated business question (severe vs. not).
- *Time variables*: year and month coded as nominal/cyclical categories rather than continuous integers, since crash risk is seasonal. Hour recoded into cyclical bands (e.g., night/day/peak) in addition to raw hour, since hour-of-day effects on severity (dark, high-speed rural roads at night) are non-linear. Day-of-week and weekend flag kept as nominal/binary. The weekend flag is partially redundant with day-of-week but retained since it is often directly what treatment/enforcement policies key off, improving interpretability even at the cost of mild redundancy.
- *Speed zone*: treated as ordinal, not purely nominal, since higher zones map to higher kinetic energy and are known from the road-safety literature to have monotonic-ish relationships with severity. This preserves interpretability of coefficients/odds ratios in a logistic model.
- *Region, urban/rural setting, road management category*: nominal, no natural order.

- *Divided-road flag, run-off-road flag*: binary/nominal.
- *Counts (total people, vehicles, heavy/passenger/motorcycle/public-transport vehicle counts)*: treated as continuous/count variables, checked for zero-inflation and skew; heavy-vehicle and public-transport counts are sparse (mostly 0) so were also converted into binary "presence" flags alongside the raw count, since presence of a heavy vehicle is known to be strongly associated with severity even at low counts, and the raw count carries little extra signal beyond "any vs none."
- *Light conditions, road layout, DCA category, accident type*: nominal categorical, encoded via one-hot or target-aware encoding depending on cardinality

**High cardinality variables**

| Variable | Issue | Treatment |
|---|---|---|
| DCA crash category | 80-100+ codes in Victorian DCA classification | *Regrouped* into smaller set of higher-level crash mechanism families using the standard DCA grouping hierarchy. Original fine-grained code retained as a secondary field for drill-down analysis |
| Accident type | Moderate to high, overlapping conceptually with DCA category | Kept at a coarser level. Cross-checked against DCA groupings for redundancy |
| Local government area | Very high cardinality | Dropped or regrouped to Region |
| Year | Moderate cardinality, potential source of temporal leakage | Kept as categorical but also inspected for trend. Used mainly for reproducibility checks (train/test split awareness) rather than as a strong predictive lever |

**Credibility, reproducibility and interpretability**

Credibility: leakage-audited feature set means model performance reflects genuinely available pre-outcome information, so reported accuracy/AUC is not artificially inflated. It is critical since the whole point is to guide real resource allocation, not to produce an impressive but useless metric.

Reproducibility: all recoding (DCA grouping, speed-zone ordinal mapping, rare-category pooling, missing-value handling) is documented as an explicit, scriptable pipeline (not ad hoc manual edits), so the same transformations can be re-applied to future crash-data extracts as the dataset is updated.

Interpretability: preference for measurement-level choices (ordinal speed zone, grouped DCA categories, presence flags for sparse vehicle types) and for a model class (e.g., logistic regression or a shallow tree/GBM with SHAP) that yields human-readable relationships. This matters because the end users are road-safety engineers and policy staff who need to translate

"high-risk profile" findings into concrete treatments (e.g., lighting upgrades, speed-zone reviews), not just a black-box severity score.

**#One limitation to flag explicitly:** this model predicts severity conditional on a crash being reported and recorded. It says nothing about *unreported* crashes or *whether* a crash occurs and should not be used to infer absolute risk at a location without pairing it with exposure/traffic-volume data.

## Exploratory assessment

**Target and input variable characteristics requiring attention**

Target imbalance: Observations in StatExplore indicate that the number of severe accidents is roughly equal to the number of normal accidents (where the `TargetSevere` variable is 1 for fatal and 0 for other, with a ratio of 4,987 to 5,013). Given that this dataset features a balanced binary classification target, I decided to use stratified splitting with a 60:20:20 ratio (Training:Validation:Test).

Mixed measurement levels among inputs:

**Target and ID**

| Variable | Role | Level | Note |
|---|---|---|---|
| **TargetSevere** | Target | Binary | 1 = fatal, 0 = other injury |
| **ACCIDENT_NO** | ID | Nominal | Unique key only. It will be excluded automatically once role = ID |

**Rejected**

| Variable | Role | Reason |
|---|---|---|
| **ACCIDENT_DATE, ACCIDENT_TIME** | Rejected | High-cardinality raw date/time. |
| **LATTITUDE, LONGTITUDE, VICGRID_X, VICGRID_Y** | Rejected | Near unique geocoordinate. Has overfitting risk. |
| **ROAD_NAME, ROAD_ROUTE_1, LGA_NAME, SRNS** | Rejected | Very high cardinality / identifier-like, redundant with coarser geography fields. |
| **DCA_CODE_DESCRIPTION** | Rejected | Text duplicate of DCA_CODE. |
| **POLICE_ATTEND** | Rejected | A leakage - attendance is structurally tied to how serious the crash looked at the scene. |
| **STAT_DIV_NAME** | Rejected | Overlap with DTP_REGION |

**Inputs – Nominal**

| Variable | Level | Note |
|---|---|---|
| **ACCIDENT_TYPE** | Nominal | No natural order |
| **DCA_CODE** | Nominal | |
| **DAY_OF_WEEK** | Nominal | Treat as Nominal, not Ordinal |
| **LIGHT_CONDITION** | Nominal | |
| **ROAD_GEOMETRY** | Nominal | |
| **ROAD_TYPE** | Nominal | |
| **RMA** | Nominal | Road management authority category |
| **DEG_URBAN_NAME** | Nominal | |
| **DTP_REGION** | Nominal | |

**Inputs – Ordinal**

| Variable | Level | Note |
|---|---|---|
| **SPEED_ZONE** | Ordinal | |

**Inputs – Binary**

| Variable | Level | Note |
|---|---|---|
| **DIVIDED** | Binary | |
| **RUN_OFFROAD** | Binary | |

**Inputs – Interval (counts)**

| Variable | Level | Note |
|---|---|---|
| **NO_OF_VEHICLES** | Interval | |
| **TOTAL_PERSONS** | Interval | |
| **MALES, FEMALES** | Interval | Person counts |
| **DRIVER, PASSENGER, PEDESTRIAN, BICYCLIST, MOTORCYCLIST, PILLION** | Interval | Person-role counts per crash |
| **HEAVYVEHICLE, MOTORCYCLE, PASSENGERVEHICLE, PT_VEHICLE** | Interval | Vehicle-type counts |
| **OLD_DRIVER_75_AND_OVER** | Interval | |
| **OLD_PED_65_AND_OVER** | Interval | |
| **PED_CYCLIST_13_18** | Interval | |
| **PED_CYCLIST_5_12** | Interval | |
| **YOUNG_DRIVER_18_25** | Interval | |

Suspected leakage variable: POLICE_ATTEND should be viewed as an information leakage risk, given that the prediction target is accident severity, yet police typically arrive at the scene when an accident is likely to be serious.

**Data-quality issues affecting modelling decisions**

High cardinality categorical: Sparse dummy variables in DCA_CODE and ROAD_TYPE can produce unstable coefficient estimates because some categories contain very few observations. Their coefficients are therefore highly sensitive to individual observations, resulting in large standard errors, high variance, and potential overfitting. Grouping them to wider categories can provide more stable estimates.

Inconsistent missing /unknown coding: "Unk.", "Unknown", "NONE" appearing as separate values across LIGHT_CONDITION, IMP_DEG_URBAN_NAME, IMP_ROAD_TYPE. These should be standardized to avoid artificially inflating category counts.

Likely data-entry artifacts: e.g., "METROPOLITAN SOUT" in IMP_DTP_REGION (truncation), directional suffixes (E, W, S) in IMP_ROAD_TYPE that look like parsing errors rather than genuine road types.

Sparse categories: MELBOURNE_CBD's low volume relative to MELB_URBAN. Then it should be merged rather than kept separate, since low counts would produce unstable model estimates for that level.

Missingness: near-negligible for most interval variables (e.g., 3/50,000). Median imputation applied for consistency rather than necessity.

**Relationships most useful for prediction or interpretation**

Relationships between crash outcomes and variables such as speed zone, road type, DCA code, time of day, and geographic characteristics are potentially useful for prediction and interpretation. Variables such as DCA_CODE may provide substantial predictive information because they describe crash circumstances. However, interpretation should be treated cautiously where categories are sparse. In particular, rare DCA_CODE and ROAD_TYPE categories can result in unstable dummy-variable estimates because their effects are based on very few observations. Therefore, relationships supported by sufficiently large groups are more reliable for interpreting general crash patterns, while predictive models can assess whether these variables improve out-of-sample performance.

**How these findings shape the next modelling stages**

- Target imbalance → stratified Data Partition, and evaluation metrics beyond raw accuracy.
- Regrouped DCA_CODE / ROAD_TYPE → these engineered variables (not the raw high-cardinality originals) go into Variable Selection.
- Zero-inflated counts → both raw count and derived presence-flag versions carried forward, letting Variable Selection (train-only) decide.
- Leakage-risk variable (POLICE_ATTEND) → excluded or tested in a sensitivity comparison.
- Standardized Unknown coding and merged sparse categories → done pre-partition so the same category structure applies consistently across train/validation/test.

**Limitations to consider when interpreting relationships**

- Associations found here are from reported crashes only. Nothing about unreported crashes or crash likelihood at a location (exposure/traffic-volume isn't in the dataset).

- Regrouping decisions (DCA families, road-type buckets) involve judgment calls that could obscure finer-grained patterns within a merged category.
- Some observed relationships may be confounded, for example speed zone correlates with road type and rural/urban setting, so simple bivariate associations may overstate an individual variable's independent effect (a reason to rely on multivariate modelling, not just EDA, for final conclusions).
- Small-sample categories (e.g., merged CBD, rare DCA codes) carry wider uncertainty even after pooling.

## Design a validation strategy

**Chosen partition strategy**

Data partition (60:20:20, stratified by TargetSevere) is applied first, immediately after Input Data. All cleaning, regrouping, and imputation steps (Impute/Replacement nodes) run downstream of partitioning.

**Justification**

Dataset size: with ~50,000 records, a three-way split is comfortably supportable. Even a 20% test partition retains enough volume (~10,000 records) for a stable estimate, unlike a small dataset where you would need to conserve Train volume more aggressively (e.g., 70:15:15).

Target imbalance: TargetSevere is much balanced (almost 50/50). This is the main driver for choosing stratified rather than simple random partitioning. Without stratification, the minority class could be unevenly distributed across partitions by chance, which would destabilize both model training and the honesty of validation or test performance estimates.

Multiple candidate models requiring fair comparison: Since I am comparing multiple model types (e.g., logistic regression, decision tree), Validation serves a distinct purpose from Test. It's used to tune and select among competing models (via a Model Comparison node), while Test remains a completely untouched holdout reserved only for the final reported performance. Using the same partition for both tuning and final reporting would bias the result upward (the model would look artificially better than it will on new data).

Leak-safe boundary: Data partition is placed before any target-informed step. This ensures Train only fitting for anything that learns from the target, consistent with the leakage principles applied in variable handling (e.g., excluding POLICE_ATTEND, dropping injury outcome fields).

Why partition-first is acceptable: None of cleaning steps (DCA regrouping, Unknown standardization, sparse categories merging like CBD => MELB_URBAN) are target-informed. They are based on category frequency and domain logic, not on TargetSevere. Since these transformations don't learn anything from the outcome, applying them after partitioning does not introduce leakage.

**Demonstrating the split is fit for purpose**

Partition sizes:

- Train: 30,000 rows
- Validation: 9999 rows

- Test: 10,001 rows

Target distribution check:

- Train: 50% of 0 and 50% of 1
- Validation: 50.005% of 0 and 49.995% of 1
- Test: 49.995% of 0 and 50.005% of 1

**How the process supports reproducibility**

Fixed random seed:

Because cleaning/grouping nodes run after partitioning and apply a single fixed rule set, the pipeline is still fully reproducible.

Single documented partition used across all models: Since every candidate model draws from the same Train/Validation/Test split rather than being independently repartitioned.

# Q2. Establishing a Baseline and Initial Candidate Model

## Set up the baseline

Baseline chosen: Logistic regression

It predicts the probability that TargetSevere = 1, via:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k$$

This is simple and easy to repeat because:

- It's a single, well-understood, closed-form estimation (maximum likelihood). It has no tuning of hyperparameters like tree depth, learning rate, or number of hidden layers, so two analysts running it on the same partitioned data get essentially the same coefficients every time.
- The model itself is fully transparent: every input's effect is a single coefficient (an odds ratio once exponentiated), so it can be reported in one table and audited line by line.
- It uses only main effects, deliberately excluding interactions or transformations, so it represents the simplest form of "a model that actually uses the crash-circumstance variables".

Why this is a fair reference point

- Same target, same partitions, same leakage rules**.** The logistic regression baseline is built using the identical Data Partition (Train/Validation/Test) and the identical leakage-audited variable set from earlier (no POLICE_ATTEND, no injury-outcome fields, no ACCIDENT_NO, no raw lat/long). So, any later model's advantage is attributable to modelling technique, not to different or cleaner inputs.
- It's the industry-standard reference class. Logistic regression is the long-standing default technique in road-safety and epidemiological crash-severity research (odds ratios for speed zone, light condition, road geometry are exactly how this literature has historically communicated findings to agencies). Comparing a neural network against logistic

regression, rather than against a trivial rule, tests whether the extra complexity earns its keep against the method the agency's own analysts could otherwise use directly.
- It uses the same measurement-level decisions already settled: SPEED_ZONE as ordinal, DCA_CODE regrouped, sparse vehicle-type variables as presence flags. So, the baseline isn't handicapped by different data prep than the more advanced models.
- It's still interpretable enough to be a decision-making tool, not just a technical hurdle. This matters because if the baseline already performs adequately, the agency may reasonably choose to deploy it directly rather than a harder-to-explain model, which is a legitimate real-world outcome of this comparison.

What the baseline can and cannot tell the agency

What it can tell:

- Genuine, ranked evidence about which recorded circumstances are statistically associated with severe outcomes (e.g., "crashes in ≥100 km/h zones have X times the odds of severe injury, holding light condition and road geometry constant"). It is directly usable for the stated purpose of guiding road treatments, enforcement and education.
- A realistic first estimate of how far the included variables get you. If a logistic model built on well-established predictors already reaches an AUC around 0.75, the agency has a credible early signal of the achievable ceiling before investing in more complex modelling infrastructure.
- An audit trail of assumptions. Because it's linear in the log-odds and uses no interactions, any large and stable AUC gain from later models are themself evidence that important interaction effects or non-linear relationships exist in the crash data (e.g., speed zone's effect on severity differing by light condition) that a simple model structurally cannot capture.

What it cannot tell:

- Interaction and non-linear effects. For example, if unlit rural roads are dramatically worse specifically at high-speed zones (an interaction), a main-effects logistic regression will understate this, since it assumes each variable's effect is constant regardless of the others' values.
- Complex, high-order variable combinations. Its patterns involving several variables simultaneously (e.g., a specific combination of ACCIDENT_TYPE + heavy-vehicle presence + rural undivided road at night) are the kind of pattern tree-based is specifically built to detect, and a linear baseline cannot represent them without manually specified interaction terms.
- A guarantee of a stable model on rare categories. With high-cardinality inputs like DCA_CODE even after regrouping, some sparse category coefficients may still be unstable (large standard errors). A limitation the baseline shares somewhat with more complex models, but one that's easier to spot and interpret in a coefficient table than buried inside an ensemble.

## Build an initial candidate model

Modelling approach: Decision Tree

A decision tree splits the crash dataset into progressively smaller, more homogeneous groups by repeatedly asking yes/no questions about the input variables (e.g., "Is REP_SPEED_ZONE above X?", "Is REP_ROAD_GEOMETRY = intersection?"). Each split is chosen to best separate severe from non-severe crashes at that point. The result is a set of rules that can be read top-to-bottom, like a flowchart, ending in a final prediction at each leaf.

Why is it a good first candidate:

- Business goal: Road-safety stakeholders need to *act* on findings. A tree produces explicit rules ("crashes on undivided roads in 100km/h zones with a heavy vehicle involved are high-risk") that map directly onto policy actions, rather than an abstract score.
- What I learned about the data: The VarClus results showed only ~26% of variation was explained by tight correlation clusters. A tree doesn't assume linearity and automatically searches for both threshold effects and interactions, so it's a good diagnostic first step even before I know the shape of the relationship.
- Explainability to a non-technical audience: Unlike a regression coefficient, which requires explaining log-odds/odds ratio, or a neural network, a tree's logic reads like a decision flowchart that a road-safety officer can follow without a statistics background.

Model settings:

- Missing value treatment: I used mean imputation to create REP_ variables rather than leaving missingness untreated. A tree with many missing values would otherwise either lose those rows or waste splits handling missingness inconsistently across variables.
- DCA_CODE_C regrouping: I reduced ~130 raw codes into a smaller number of meaningful categories. This matters because a nominal variable with 130 levels would force the tree into extremely fragmented.
- Maximum branch = 2, maximum depth = 6: Binary splits keep the tree readable as a set of simple rules; depth 6 caps how many conditions stack up in one rule, keeping it interpretable for a non-technical audience rather than producing 15-condition rules no one can act on.
- Leaf size raised to 50: With a leaf size of 5, the tree could carve out tiny terminal nodes representing a handful of crashes. Raising it to 50 forces each rule to be backed by a reasonably sized group of crashes, trading a small amount of fit for rules that are safe to generalise from.
- Subtree method = Assessment: The tree is grown fully then pruned back based on Validation performance, not Training fit alone.

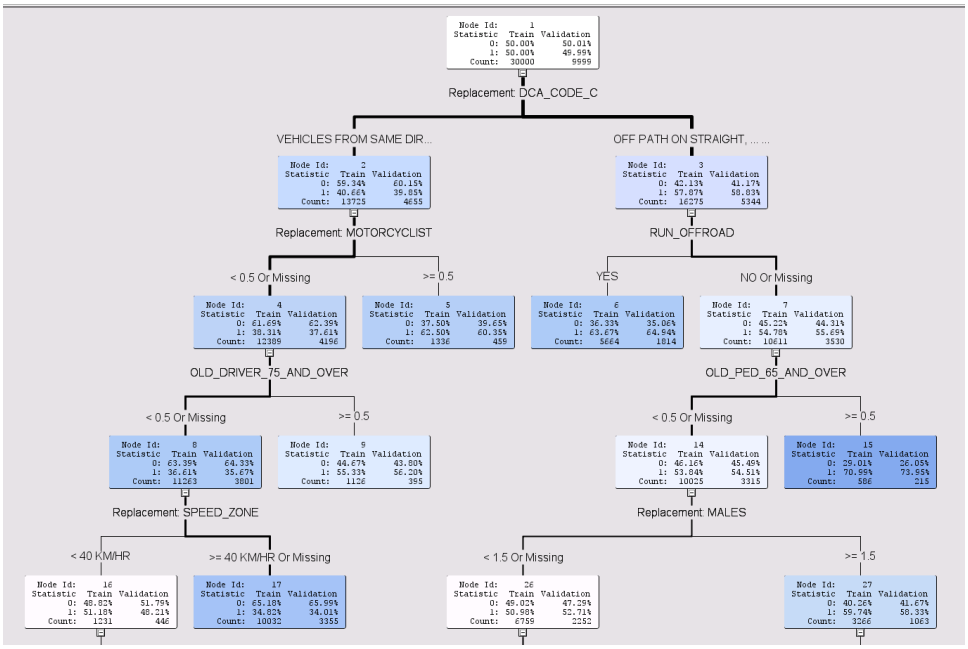
Model output to show:

Variables: Keep variables same as Logistic regression.

Decision tree’s node properties: Leaf size = 50, Maximum depth = 6, Subtree method = Assessment, Missing values = Use in search.

Fit statistics (misclassification rate): Train 0.38, Validation 0.38, Test 0.39 => The model is not overfitting.

The tree diagram (first 4 splits):




Comparison table:

Fit Statistics
Model Selection based on Valid: Misclassification Rate (_VMISC_)

| Selected Model | Model Node | Model Description | Valid: Misclassification Rate | Train: Average Squared Error | Train: Misclassification Rate | Valid: Average Squared Error |
|---|---|---|---|---|---|---|
| Y | Reg | Regression | 0.36534 | 0.22492 | 0.37100 | 0.22509 |
| | Tree | Decision Tree | 0.38104 | 0.23450 | 0.38777 | 0.23323 |

<u>Important findings in first 4 splits of the decision tree:</u>

The three's first split separated crashes by contributing factor code. Crashes classified as 'Off path on straight' showed a higher proportion of severe outcomes (57.87% in training) than crashes classified as 'Vehicles from same direction' (40.66%).

Within the 'Vehicles from same direction' group, crashes involving a motorcyclist had a notably higher severe rate (62.5%) than those without (38.31%). This likely reflects the reduced physical protection for motorcyclists in a collision rather than motorcyclists involvement causing more severe crash dynamics.

Within the 'Off path on straight' group, crashes where the vehicle ran off-road had a higher severe rate (63.67%) than those that did not (44.31%). Running off-road is often associated with higher speed or driver distraction. The tree cannot distinguish whether off-road departure itself or an underlying factor like speed that leads to both departure and severity is doing the work.

The tree further split on driver age (75+) on the left branch and pedestrian age (65+) on the right branch, with each older-age group showing a higher severe rate than the corresponding younger group. Older drivers/pedestrians may face higher severity due to physical frailty rather than being involved in objectively worse crashes. So, this split may reflect physical vulnerability to injury at a given crash severity rather than older road users being involved in more severe crash events.

On the motorcyclist-involved branch, speed zones of 40km/h or higher were associated with a higher severe rate (65.99%) than lower zones (48.94%). Consistent with known relationships between travel speed and crash severity, though this single split cannot isolate speed's independent effect from other correlated factors on that branch.

*#Note: Several splits use REP_ variables that were mean/mode-imputed for missing values, and the tree explicitly grouped 'Missing' with one branch (e.g., MOTORCYCLIST < 0.5 Or Missing). Where missingness is folded into a branch, that branch's severity rate reflects a mix of confirmed and imputed/unknown cases, which is worth noting as a limitation rather than a substantive finding about the variable itself.*

## Decide whether the candidate improves on the baseline

Decision Tree vs Logistic Regression (baseline):

| Model | Train | Validation | Test |
|---|---|---|---|
| **Logistic Regression (baseline)** | 0.37 | 0.36 | 0.36 |
| **Decision Tree (candidate)** | 0.38 | 0.38 | 0.39 |

Model evaluation beyond Misclassification Rate:

On the primary measure, the candidate Decision Tree does not improve on the baseline. It is 2 percentage points worse (0.38 vs 0.36). Both still beat the ~0.50 error on this roughly balanced target so the tree has learned real signal but has not learned it better than the linear model.

There are some other validation measures to look on:

```
Event Classification Table
Model Selection based on Valid: Misclassification Rate (_VMISC_)

               Model     Data                  Target   False    True     False    True
Model Node  Description  Role        Target    Label  Negative Negative Positive Positive

Tree       Decision Tree TRAIN    TargetSevere          4448     7815     7185     10552
Tree       Decision Tree VALIDATE TargetSevere          1460     2650     2350      3539
Reg        Regression    TRAIN    TargetSevere          5591     9461     5539      9409
Reg        Regression    VALIDATE TargetSevere          1854     3201     1799      3145
```

| Metric | Decision Tree | Regression |
|---|---|---|
| **Recall** | 0.708 | 0.629 |
| **Precision** | 0.601 | 0.636 |
| **F1** | 0.530 | 0.633 |

*Why Misclassification Rate alone is misleading:* Misclassification rate treats false negative (a severe crash predicted as not severe) and a false positive (a non-severe crash predicted as severe) as equally costly. For this task, they are not: a missed severe crash represents a lost opportunity for road-safety intervention, while a false alarm only costs some misdirected attention. If judged poorly on misclassification, Logistic Regression appears superior (0.36 vs 0.38). However, this measure obscures a more important difference in how the two models behave with respect to outcome that matters most operationally.

*Recall as the relevant metric:* Recall measures the proportion of severe crashes each model successfully identifies. Here, the Decision Tree achieves substantially higher Recall (0.708) than Regression (0.629). It correctly flags roughly 71% of severe crashes, compared to 63% for Regression. This comes at the cost of more false positives, which is reflected in its slightly lower Precision (0.601 vs 0.636) and higher overall misclassification rate. Given that the cost of missing a severe crash is higher than the cost of a false alarm in a road-safety context, this trade-off favours the Decision Tree.

*F1 as a Check against over-prediction:* F1 balances Precision and Recall, guarding against the trivial case where a model achieves high Recall simply by over-predicting the positive class. The Tree's F1 (0.650) is still higher than Regression's (0.633), confirming that its Recall advantage is not solely the result of indiscriminate over-flagging, but reflects a genuinely better-balanced identification of severe crashes.

*Revised model recommendation:* Based on Recall and F1, the Decision Tree is the more suitable model, despite its higher misclassification rate. This illustrates why model selection should be driven by task-appropriate metrics rather than overall accuracy alone: a model that is marginally "more often correct" can still be the weaker choice if its errors are concentrated in the outcome that matters most.

*#Note: This comparison uses the default 0.5 classification cutoff for both models. Since Recall is prioritised here, testing lower thresholds may further improve Recall at some cost to Precision.*

Does the gain justify the added complexity?

The Decision Tree's advantage isn't trivial: a Recall improvement from 0.629 to 0.708 means it catches roughly 80 more severe crashes per 1,000 severe cases than Regression does. Set against that, the complexity cost of the Tree here is modest, not the steep kind you'd pay with an ensemble or neural network:

- It's still a single, readable structure.
- It required no additional feature engineering beyond what Regression already used.
- The computational cost is negligible at this data size.

Weakness/Uncertainty to guide the Next stage:

The precision–recall trade-off has only been evaluated at the default 0.5 cutoff, and no explicit cost ratio between a missed severe crash and a false alarm has been established.

The entire case for preferring the Tree rests on valuing Recall over Precision, but how much more that missed severe crash should be weighed against a false alarm hasn't been quantified. Without that, "the Tree is more suitable" is a defensible judgment call, not a rigorously optimised one. The next stage of modelling should:

- Define or approximate a cost ratio in consultation with a road-safety stakeholder.
- Use that ratio in a cost/profit matrix (via a Decision node in EM) to find the threshold, for both Tree and Regression, that minimises expected cost rather than relying on the default 0.5 cutoff.
- Rerun the Recall/Precision/F1 comparison at that optimised threshold, since the current advantage attributed to the Tree may shrink, disappear, or even reverse once Regression

is also evaluated at a cutoff tuned to the same cost assumption, rather than comparing one model's default behaviour against another's.

# Q3. Developing a Contrasting Modelling Strategy

## Choose a different model family

Second model candidate: Random Forest

The single Decision Tree was deliberately constrained specifically to keep it stable and readable. Those same constraints mean it can only ask a limited number of sequential questions before stopping, so it may miss weaker interaction effects or alternative splitting paths that a deeper tree would have found. Logistic Regression, meanwhile, assumes each predictor contributes additively unless you build that interaction in by hand.

Random Forest addresses both gaps differently from either prior model: it grows a large number of largely uncorrelated trees, each trained on a random subset of crashes and a random subset of variables at each split, then combines their predictions by averaging or voting. Because each tree in the forest is free to find a different sequence of splits, the ensemble can surface interaction effects and non-linear thresholds that a single constrained tree pruned early, without requiring you to specify them in advance the way Regression does. It effectively runs many alternative "hypotheses" about how the predictors combine, rather than committing to one tree's specific path.

Assumptions, data needs, and practical limits:

- Fewer distributional assumptions than Regression. Random Forest makes no assumption of linearity or a particular error distribution, which suits my earlier finding that predictors are largely uncorrelated and possibly non-linear in effect.
- Larger size dataset needed. With ~50,000 rows and a roughly balanced target, the dataset is large enough to support building and averaging many trees without each one being trained on too few observations.
- Handles the mixed variable types I already have without needing the re-encoding that a distance-based method like KNN would require.
- Practical limits: it is heavier than a single Tree or Regression because of training hundreds of trees takes longer and produces a larger model object. So it needs reasonable defaults rather than being left entirely automatic, or it will risk either under-fitting or unnecessary runtime cost.

Interpretability and complexity compared with the baseline and first candidate:

| | **Logistic Regression (baseline)** | **Decision Tree (1st candidate)** | **Random Forest (2nd candidate)** |
|---|---|---|---|
| **Interpretability** | High | Moderate | Low |
| **Complexity** | Low | Low | High |
| **Captures non-linearity or interactions** | Only if specified manually | Yes, but limited by depth/leaf-size constraints | Yes, more thoroughly via many independent trees |

Random Forest is clearly the least interpretable of the three: there is no single path or coefficient to point to, since the prediction for any crash reflects a vote or average across hundreds of trees. It partially compensates for this with a Variable Importance ranking, which shows which predictors most reduced prediction error across the forest. This places it a step below the single Tree in explainability and well below Regression.

Why this model is suitable for the intended prediction point and road-safety context:

The prediction is made before or shortly after a crash is recorded, using pre-crash and crash-recording variables (speed zone, road geometry, vehicle types, persons involved, contributing-factor codes) to flag likely severity. Random Forest is well suited to this stage precisely because it does not require that assumption: it can absorb whatever combination of thresholds and interactions the data contains without the analyst needing to have already guessed the correct functional form.

For the road-safety context specifically, Random Forest is proposed here as an evidence-strengthening check, not a replacement for the interpretable models: if its predictive performance (particularly Recall, given the earlier reasoning that missed severe crashes are the costlier error) closely matches the Tree and Regression, that convergence across a third, structurally different method further supports the conclusion that the ceiling reflects genuine data limits rather than a weakness specific to any one model. If Random Forest instead shows a clear improvement, that will indicate real non-linear or interaction signal was left on the table by the simpler models, and its Variable Importance output can still guide which factors deserve attention.

## Configure and assess the contrasting model

Design choice: what should vary and why

The two settings most likely to meaningfully change a Random Forest's flexibility are Number of variables considered at each split (m) and tree depth / minimum leaf size. These directly control how much freedom each individual tree has, and how much the trees in the forest resemble each other:

- m (variables per split) controls how different trees are from one another. A small m forces more randomness and variety across trees (each tree "sees" a narrower slice of predictors at each decision point). A large m (approaching total variable count) makes trees more like each other and closer in behaviour to a single and unconstrained Tree.
- Leaf size / depth controls how much each individual tree can fit to the training data before stopping. Small leaves let trees grow deep and fit fine-grained patterns (higher variance per tree). Large leaves keep trees shallower and more stable (higher bias per tree).

⇨ Varying these together produces two configurations that sit at genuinely different points on the flexibility spectrum, rather than two versions that are trivially similar.

Configuration A: Conservative / Stable Forest

Settings:

- Number of variables to consider (m): 5
- Minimum leaf size: 10

- Number of trees: 100
- Maximum Depth: 50

*#Note: This is close to standard Random Forest practice. It has enough per-split randomness to decorrelate trees but leaves large enough that no single tree memorises noise from a handful of observations.*

Configuration B: Flexible / High-variance Forest

Settings:

- Number of variables to consider (m): 2
- Minimum leaf size: 1
- Number of trees: 100
- Maximum Depth: 50

*#Note: Lowering m increases the diversity across trees (more randomness at each split), while allowing leaf size down to 1 lets each individual tree fit very fine-grained, potentially noisy patterns. This configuration tests the opposite end of the flexibility spectrum from Configuration A.*

#Results:

| | **Configuration A** | **Configuration B** |
|---|---|---|
| **m (vars/split)** | 5 | 2 |
| **Min leaf size** | 10 | 1 |
| **Train** | 0.36 | 0.38 |
| **Validation** | 0.36 | 0.37 |
| **Test** | 0.37 | 0.38 |
| **Train-Validation gap** | 0.00 | -0.01 |

Effect on flexibility, stability and over/under-fitting

Neither configuration shows over-fitting. In both cases, Train and Validation misclassification are almost identical (A: 0.36/0.36; B: 0.38/0.37). The near-zero gap in both configurations indicates the forest's averaging across 100 trees is successfully controlling variance regardless of the individual tree settings used. This is one of Random Forest's core strengths: because predictions are averaged across many trees built on different bootstrap samples, even a more flexible individual-tree setup doesn't easily translate into the classic over-fitting signature of much lower Train error than Validation error.

Configuration B is weaker, but for an under-fitting reason, not the one initially expected. The original intent behind Configuration B (m=2, minimum leaf size=1) was to increase flexibility. Smaller leaves should let individual trees fit more closely to the training data, typically lowering Train error compared to a more constrained setup. Instead, Train error is higher in Configuration B (0.38) than Configuration A (0.36). This points to the m setting (variables considered per split) dominating the leaf-size effect: with only 2 candidate variables considered at each split, and the input set already shown to be largely weakly correlated with each other, many splits in Configuration B are choosing from a pair of variables that includes little genuinely useful signal. Allowing very small leaves doesn't help

if the splits used to reach them weren't well chosen in the first place. The trees end up both noisier and weaker, rather than more expressive. This is effectively an under-fitting problem introduced by starving each split of enough informative candidate variables, not the over-fitting problem that smaller leaves = more flexible would normally predict.

Configuration A's settings are better balanced for this dataset. With m=5 and a leaf-size floor of 10, each split has enough candidate variables to find a genuinely useful one, while the leaf-size floor prevents any single tree from fitting down to noise-level groups of observations. The result is both lower error and a stable Train–Validation gap.

Identify the stronger version

Model selection here should be based on Validation results, not Training results alone, since Training performance does not indicate how a model will generalise to new crashes. On Validation misclassification, Configuration A (0.36) outperforms Configuration B (0.37). Because Configuration A wins on Validation and shows no evidence of over-fitting (Train ≈ Validation), Configuration A is the stronger version and should be carried forward as the tuned Random Forest for comparison against the Decision Tree and Logistic Regression baseline.

## Compare the modelling strategies

Primary validation measure: Misclassification rate

| Model | Train | Validation | Test |
|---|---|---|---|
| **Logistic Regression (baseline)** | 0.37 | 0.36 | 0.36 |
| **Decision Tree (1st candidate)** | 0.38 | 0.38 | 0.39 |
| **Random Forest (2nd candidate)** | 0.36 | 0.36 | 0.37 |

On the primary measure, Random Forest matches the baseline exactly on Validation (0.36) and improves on the single Decision Tree (0.38). It is also the most stable of the three across Train/Validation/Test, with the smallest spread between partitions.

Supporting measures: Recall, Precision, F1 (Validation)

Event Classification Table

Data Role=TRAIN Target=TargetSevere Target Label=' '

| False Negative | True Negative | False Positive | True Positive |
|---|---|---|---|
| 5124 | 9298 | 5702 | 9876 |

Data Role=VALIDATE Target=TargetSevere Target Label=' '

| False Negative | True Negative | False Positive | True Positive |
|---|---|---|---|
| 1735 | 3086 | 1914 | 3264 |

| Metric | Regression | Decision Tree | Random Forest |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **Recall** | 0.629 | 0.708 | 0.653 |
| **Precision** | 0.636 | 0.601 | 0.630 |
| **F1** | 0.633 | 0.650 | 0.641 |

These reveal a trade-off invisible in misclassification alone. The Tree catches the most severe crashes (highest Recall) but at the largest Precision cost. Random Forest sits between the two: a meaningful Recall improvement over Regression (0.653 vs 0.629) while barely sacrificing Precision (0.630 vs 0.636). We can see this is a more efficient trade-off than the Tree's.

Meaningful improvement or small numerical difference

On raw misclassification, Random Forest's result is not a meaningful improvement over Regression. If framed only by the primary measure, switching from Regression to Random Forest would not be justified.

However, because I framed them by Recall, the measure more aligned with the road-safety objective of catching severe crashes, Random Forest does offer a real improvement over Regression (+2.4 percentage points). It achieved without the steep Precision trade-off the Tree required for its larger Recall gain. This is a genuine and relevant improvement, not just noise, because it moves in the same direction as the Tree's earlier finding while avoiding the Tree's largest weakness.

Trade-off: Predictive Performance, Complexity, Stability, Interpretability

| | **Regression** | **Decision Tree** | **Random Forest** |
|---|---|---|---|
| **Predictive performance** | Baseline | Weaker on accuracy, strongest Recall | Matches baseline accuracy, second-best Recall/F1 |
| **Complexity** | Low | Low | High |
| **Stability** | Stable | Stable | Most stable of the three |
| **Interpretability** | High | Moderate | Low |

The clearest trade-off is between interpretability and the Recall/stability gain: Random Forest is the least explainable of the three to a non-technical audience, yet it delivers the most stable performance across partitions and a better-balanced Recall/Precision trade-off than either simpler model. Regression remains the easiest to explain and justify to stakeholders (clear direction and magnitude per variable) but leaves some Recall on the table. The single Tree sits at neither extreme: more explainable than Random Forest, more Recall than Regression, but the least stable/accurate of the three and prone to a steeper Precision cost.

Provisional decision

I decided to carry forward Random Forest as the working model at this stage, with Logistic Regression retained as the interpretable reference model for the following reasons:

- It matches the baseline on overall accuracy (0.36), so choosing it costs nothing on the primary measure.
- It improves meaningfully on Recall relative to Regression, better serving the stated goal of catching severe crashes, without the Tree's larger Precision trade-off.
- It shows the best stability across Train/Validation/Test of the three models, reducing concern about the result being partition specific.

- Its main cost is judged acceptable at this provisional stage, since Variable Importance still supports a "what matters most" narrative for stakeholders, even without a specific rule to quote.

## Translate the findings for decision-makers

What this model adds to the road-safety problem

This model looks at the details recorded about a crash and estimates how likely that crash is to be classified as severe. It doesn't replace judgement or investigation, but it gives the agency a way to flag which crashes, or which combinations of conditions, deserve closer attention rather than treating every crash report the same.

The model consistently points to a small set of factors as the strongest indicators of severity: Speed zone, Road geometry/type, Vehicle mix, and Crash mechanism (DCA code regrouping). Put together this paints a risk profile: a higher-risk crash pattern tends to combine higher speed zones, certain road layouts, and particular crash mechanisms.

How this supports investigation and resource prioritisation

Thes patterns can guide where attention and resources are directed without needing to review every crash record manually:

- Targeted enforcement or engineering review. Locations or road types that consistently show up in the model's high-risk pattern could be prioritised for a closer safety audit, signage review or speed management measures.
- Triage for further investigation. New crash reports flagged reports flagged by the model a high-risk for severity could be reviewed sooner or in more depth than lower-risk reports, which make better use of limited investigative time.
- Monitoring emerging patterns. Tracking which factors the model relies on over time can help the agency notice if a particular risk combination is becoming more common and prompting a proactive rather than reactive response.

Why this is not proof of cause

It's important to be clear that the model shows association, not causation. For example, if crashes in a certain speed zone are more often severe, that does not necessarily mean the speed zone itself is the direct cause. It could be that this speed zone is also associated with other factors the model doesn't fully capture, such as typical vehicle speeds travelled (as opposed to the posted limit), road users' familiarity with the area, or the mix of vehicle types that commonly use that road. The model identifies patterns that hold across many recorded crashes; it cannot, on its own, establish that changing one factor (e.g., lowering a speed limit) would directly reduce severity, since that would require a controlled study or intervention, not just an analysis of historical records.

One limitation affecting responsible use

The model's predictions are only as good as the historical, recorded circumstances captured in the dataset. It has no information about factors that aren't recorded, such as the exact dynamics of impact, whether safety equipment was used correctly, or split-second decisions made at the time. Because of this, the model should be used to support and prioritise human

judgement and investigation, not to make final determinations about a crash's severity or to justify decisions (such as resourcing or enforcement priorities) without human review. It I s particularly since all three model types tested converged on a similar level of accuracy, indicating a meaningful share of crash severity is not explained by the variables available to the model.

# Q4. Challenging and Refining the Modelling Strategy

## Diagnose a weakness in the current modelling strategy

The models disagree on which error to make and that disagreement has not yet been resolved by an explicit cost decision.

The Decision Tree catches noticeably more genuine severe crashes than Regression but does so at the cost of more false alarms and a worse overall misclassification rate. Random Forest sits between the two on every measure, improving on Regression's Recall while narrowing but not closing the Tree's advantage.

Every threshold-based conclusion drawn so far rests on an *assumed* relative cost between missing a severe crash and issuing a false alarm, not established through an actual cost figure from a road-safety stakeholder or a formal cost matrix run through a Decision node. Without that, it is not possible to say definitively whether the Tree's Recall gain is worth its Precision cost, or whether Random Forest's more balanced trade-off is the better operating point.

The three models were built on identical inputs and partitions yet converge to similar misclassification (~0.36–0.39) while diverging meaningfully on Recall (0.629–0.708). This spread shows the models are making genuinely different trade-offs between the two error types, not just noisy variations of the same result.

## Design and implement a defensible extension

The question this improvement is meant to answer:

*Is the model ranking established so far (Regression vs Tree vs Random Forest) an artefact of comparing all three at an arbitrary default cutoff, or does one model still come out ahead once each is evaluated at a threshold that reflects the actual relative cost of missing a severe crash versus raising a false alarm?*

Why this addresses the identified weakness:

The weakness identified was that Regression, Tree, and Random Forest were compared using each model's default 0.5 classification cutoff. That assumption was never actually justified. It was simply the default. Since the three models showed materially different Recall/Precision trade-offs at that shared default, the model ranking is sensitive to a threshold choice that has not yet been deliberately set. Refining how the model is selected directly resolves this, rather than leaving the comparison dependent on an unexamined default.

Implementation decisions

- Change type: this is a refinement to model selection, not a new algorithm
- Mechanism: using cost/profit matrix rather than default.

- Assumed cost ratio: a missed severe crash was weighted as 3 times as costly as a false alarm. This ratio is an assumption, made explicit here and flagged as provisional pending confirmation from a road-safety stakeholder, rather than a false precision.
- The same models, the same REP_ variable set, and the same Validation partition will stay constant.

Event Classification Table
Model Selection based on Valid: Misclassification Rate (_VMISC_)

| Model Node | Model Description | Data Role | Target | Target Label | False Negative | True Negative | False Positive | True Positive |
|---|---|---|---|---|---|---|---|---|
| Dec3 | Decisions | TRAIN | TargetSevere | | 5591 | 9461 | 5539 | 9409 |
| Dec3 | Decisions | VALIDATE | TargetSevere | | 1854 | 3201 | 1799 | 3145 |
| Dec4 | Decisions | TRAIN | TargetSevere | | 4448 | 7815 | 7185 | 10552 |
| Dec4 | Decisions | VALIDATE | TargetSevere | | 1460 | 2650 | 2350 | 3539 |
| Dec5 | Decisions | TRAIN | TargetSevere | | 5306 | 9418 | 5582 | 9694 |
| Dec5 | Decisions | VALIDATE | TargetSevere | | 1826 | 3119 | 1881 | 3173 |

Applying a 3:1 cost weighting between a missed severe crash and a false alarm produced no change in the optimal classification cutoff and resulting Recall/Precision trade-off for models. This suggests the models' predicted probabilities are not highly sensitive to this cost assumption within a plausible range and that the earlier comparison at the default 0.5 cutoff was a reasonably stable reflection of each model's trad-off. This increases confidence in the original model comparison even though the cost ratio itself remains an assumption pending stakeholder confirmation.

*#Note: At an extreme 10:1 ratio, Recall for Random Forest shifted to 0.635, confirming that the cutoff does respond to sufficiently large cost differences but that a moderate, more defensible ratio (3:1) does not materially alter the original ranking.*

## Judge whether the improvement is worth retaining

Random Forest is carried forward as the leading candidate for the final comparison. It matches the logistic regression baseline on overall misclassification, achieves the best balance of Recall and Precision among the three models tested, and its relative standing was not undermined by a cost-sensitivity check using an assumed 3:1 weighting between a missed severe crash and a false alarm. This decision remains provisional: it depends on an assumed rather than confirmed cost ratio, and should be revisited once the agency provides an actual estimate of the relative costs involved.

# Q5. Model Comparison, Scoring and Recommendation

## Compare all models

| Model | Validation Misclassification | Validation Recall | Validation Precision | Validation F1 |
|---|---|---|---|---|
| **Logistics Regression (baseline)** | 0.36 | 0.629 | 0.636 | 0.633 |
| **Decision Tree** | 0.38 | 0.708 | 0.601 | 0.650 |
| **Random Forest (carried forward)** | 0.36 | 0.653 | 0.630 | 0.641 |

The baseline already performs credibly ((0.36 misclassification, well above a ~0.50 random-guess floor given the near-balanced target), so any candidate needs to be judged against a genuinely competent starting point. The Decision Tree does not improve on the baseline's misclassification and only exceeds it on Recall, at a real Precision cost. Random Forest is the only candidate that matches the baseline's misclassification while also improving its Recall and F1.

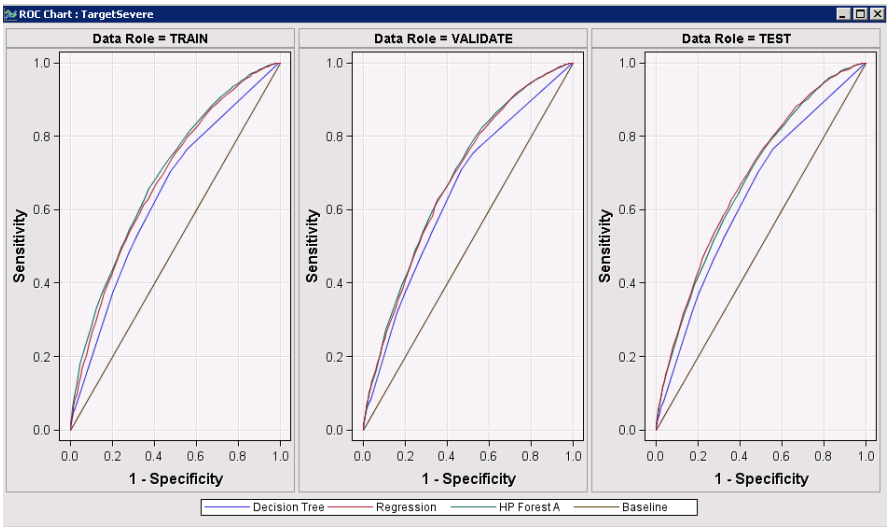
⇨ Random Forest is identified as the best model based on Validation results: it ties the baseline on the primary measure (misclassification, 0.36) and achieves the highest F1 (0.641) of all three models, indicating the best overall balance between catching severe crashes and avoiding false alarms.

Justifying the choice of statistics

F1, alongside misclassification, was used as the deciding statistic rather than misclassification alone, because misclassification treats a missed severe crash (false negative) and a false alarm (false positive) as equally costly. F1 balances Recall (the task-relevant "catch rate" for severe crashes) against Precision (avoiding excessive false alarms), so a model cannot score well on F1 simply by over- or under-predicting the positive class. Using F1 as a secondary decision statistic, alongside the primary misclassification measure, keeps the choice grounded in the stated business goal rather than a single generic accuracy figure.

Trade-off:

- Discrimination (ROC):




- Classification performance: Random Forest offers the best balance
- Interpretability: Regression < Decision Tree < Random Forest
- Complexity: Regression < Decision Tree < Random Forest

The trad-off is real: Random Forest's performance edge comes at the cost of being the least transparent of the three to a road-safety audience. This is why the final recommendation is Random Forest because it is preferred for predictive performance, but its Variable Importance

output should be paired with the more interpretable Regression or Tree outputs when explaining why a crash was flagged, rather than presenting Random Forest as a stand-alone and self-explanatory answer.

Why comparing different approaches is better than one model family:

Testing a linear model (Regression), a single rule-based model (Tree), and an ensemble (Random Forest) provides stronger evidence than relying on one family alone:

- If only Regression had been tested, there would be no way to know whether its ~0.36 misclassification reflects the real limit of what the data can predict, or simply a limitation of assuming linear/additive relationships.
- The fact that all three converge to a similar performance ceiling (0.36–0.39), despite using fundamentally different mechanisms (a linear equation, sequential rule-splitting, and an averaged ensemble of many trees), is itself important evidence: it suggests the ceiling reflects a genuine limit in what the available variables can explain about crash severity, rather than a weakness specific to anyone modelling approach. A single-family comparison could never have supported that conclusion.

## Score the selected model

What Scoring means in SAS Enterprise Miner

Scoring is the process of applying an already built and validated model to new data that was not used to train it, in order to generate a prediction for each record. In this case, the model is applied to Victoria_Crash_Test.csv, a separate file the model has never seen before. For each crash record, scoring produces two things: a predicted probability that the crash belongs to the high severity category, and a predicted classification based on that probability and a chosen cutoff. Scoring does not re-fit or change the model in any way.

Because the model outputs a continuous predicted probability rather than only a binary label, records can be ranked from highest to lowest predicted risk. This ranking allows the agency to prioritise its limited investigative or preventive resources toward the crashes and conditions most associated with severe outcomes. For example, directing engineering review toward road segments that repeatedly appear in the highest-risk band, or triaging newly reported crashes for faster follow-up when they score highly. Rather than treating every record identically, the ranked output provides a practical way to allocate attention where the recorded patterns suggest it is most warranted.

A high predicted probability reflects that a crash shares characteristics with historically severe crashes in the training data, it is not a certainty, and some crashes with high predicted probabilities will not, in fact, be severe. The model also identifies statistical association between input variables and outcome severity; it does not establish that any variable causally determines severity. Other unmeasured factors may explain part of the pattern, and any resourcing or policy action based on these results should be treated as a starting point for further investigation, not as a final determination of risk or cause.

## Business recommendation

Recommended model: Random Forest

Random Forest is recommended as the model to take forward for identifying crashes likely to be high severity. On the Validation data Random Forest matched the best-performing simpler model (Logistic Regression) on overall accuracy (36% error rate) while achieving the best balance between correctly catching severe crashes and avoiding false alarms (highest F1 score of the three approaches tested). A single Decision Tree caught slightly more severe crashes, but at the cost of a noticeably higher false-alarm rate; Random Forest offers the strongest all-round trade-off between these two error types across the three approaches tested. Its advantage also proved stable when re-checked under different assumptions about how costly a missed severe crash is compared with a false alarm.

The model should be used to rank and prioritise, not to make final decisions on its own. Its predicted risk scores can help direct limited investigative or engineering resources toward crash types, locations, or conditions that most resemble historically severe crashes. For example, flagging combinations of higher speed zones, particular road layouts, and certain crash mechanisms for closer review. It should support human judgement and existing investigation processes, not replace them, and predictions should never be treated as a certain outcome for any individual case.

Limitations and cautions

1. The dataset contains only recorded crashes, not exposure data. The model has learned patterns from crashes that occurred and were reported. This means the model can highlight which recorded crashes look like past severe ones, but it cannot say how likely a crash is to occur in the first place, or measure risk relative to overall traffic volume or exposure on a given road.
2. Association, not causation. The model identifies statistical patterns linking certain conditions (e.g., speed zone, road type) to severity in the historical data. It does not prove that changing any one of these factors would directly reduce severity.

Recommended monitoring step

Before relying on this model operationally, and periodically afterward, the agency should re-validate the model's performance on a rolling sample of newly recorded crashes (e.g., quarterly), comparing predicted risk classifications against actual recorded outcomes. This checks whether the model's accuracy holds up as road conditions, vehicle mix, or reporting practices change over time, and provides an early warning if the model's predictions begin to drift away from real-world outcomes, at which point retraining should be considered.